25.2 Canvas Host 实现

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~185k tokens，输出 ~6k tokens（本节）

上一节介绍了 Canvas 和 A2UI 的概念。这一节深入 Canvas Host 的源码实现，分三个部分：Canvas 静态文件服务器、A2UI 核心模块、以及 A2UI 打包流程。

25.2.1 Canvas 服务器（`src/canvas-host/server.ts`）

两层架构：Handler + Server

Canvas Host 的实现分为两层：

```
startCanvasHost()          ← 外层：创建 HTTP 服务器，绑定端口
  └── createCanvasHostHandler()  ← 内层：文件服务逻辑、Live Reload
```

这种分离允许 Canvas Handler 被**嵌入到已有的 HTTP 服务器**中（例如 Gateway 的 HTTP 层），而不必总是启动独立的服务器。

```typescript
// src/canvas-host/server.ts — 类型定义

export type CanvasHostHandler = {
  rootDir: string;        // Canvas 根目录
  basePath: string;       // URL 基路径（默认 /__openclaw__/canvas）
  handleHttpRequest: (req, res) => Promise<boolean>;  // HTTP 请求处理
  handleUpgrade: (req, socket, head) => boolean;      // WebSocket 升级处理
  close: () => Promise<void>;                         // 清理资源
};

export type CanvasHostServer = {
  port: number;           // 监听端口
  rootDir: string;        // Canvas 根目录
  close: () => Promise<void>;
};
```

Canvas 根目录准备

Canvas 的内容目录默认位于 `~/.openclaw/canvas/`：

```typescript
// src/canvas-host/server.ts — resolveDefaultCanvasRoot

function resolveDefaultCanvasRoot(): string {
  const candidates = [path.join(os.homedir(), ".openclaw", "canvas")];
  const existing = candidates.find(dir => {
    try { return fsSync.statSync(dir).isDirectory(); }
    catch { return false; }
  });
  return existing ?? candidates[0];
}
```

服务器启动时会确保该目录存在，并在缺少 `index.html` 时写入一个默认的测试页面：

```
// src/canvas-host/server.ts — prepareCanvasRoot

async function prepareCanvasRoot(rootDir: string) {
  await ensureDir(rootDir);                       // 确保目录存在
  const rootReal = await fs.realpath(rootDir);    // 解析符号链接
  try {
    await fs.stat(path.join(rootReal, "index.html"));
  } catch {
    // 不存在 index.html，写入默认测试页面
    await fs.writeFile(
      path.join(rootReal, "index.html"),
      defaultIndexHTML(),  // 包含 Hello/Time/Photo/Dalek 测试按钮
      "utf8"
    );
  }
  return rootReal;
}
```

默认测试页面 `defaultIndexHTML()` 是一个内嵌的完整 HTML 字符串，包含四个按钮（Hello、Time、Photo、Dalek），用于验证 Canvas 和 A2UI 动作桥是否正常工作。

文件服务与路径安全

Canvas Host 的核心是一个静态文件服务器，但它必须防止**路径遍历攻击**（Path Traversal）——恶意请求通过 `../../etc/passwd` 这样的路径试图访问根目录以外的文件。

```ts
// src/canvas-host/server.ts — resolveFilePath（简化版）

async function resolveFilePath(rootReal: string, urlPath: string) {
  const normalized = normalizeUrlPath(urlPath);
  const rel = normalized.replace(/^\/+/, "");

  // 第一道防线：拒绝包含 ".." 的路径段
  if (rel.split("/").some(p => p === "..")) {
    return null;
  }

  // 使用安全的文件打开函数
  const tryOpen = async (relative: string) => {
    try {
      return await openFileWithinRoot({ rootDir: rootReal, relativePath: relative });
    } catch (err) {
      if (err instanceof SafeOpenError) return null;
      throw err;
    }
  };

  // 目录请求 → 自动查找 index.html
  if (normalized.endsWith("/")) {
    return await tryOpen(path.posix.join(rel, "index.html"));
  }

  // 检查是否为符号链接（拒绝）或目录（查找 index.html）
  const candidate = path.join(rootReal, rel);
  const st = await fs.lstat(candidate);
  if (st.isSymbolicLink()) return null;   // 拒绝符号链接
  if (st.isDirectory()) {
    return await tryOpen(path.posix.join(rel, "index.html"));
  }

  return await tryOpen(rel);
}
```

路径安全通过三重机制保障：

层级	机制	说明
第一层	URL 规范化	`decodeURIComponent` + `path.posix.normalize` 消除编码绕过
第二层	`..` 段检测	显式拒绝包含 `..` 的路径段
第三层	`openFileWithinRoot`	底层安全函数（来自 `src/infra/fs-safe.ts`），通过 `realpath` 验证最终路径确实在根目录内

> **衍生解释——路径遍历攻击（Path Traversal）**：这是一种常见的 Web 安全漏洞。攻击者通过在 URL 中使用 `../`（父目录引用）来尝试访问服务器上预期目录之外的文件。例如，请求 `http://server/files/../../../etc/passwd` 试图读取系统密码文件。防御方法包括：规范化路径、过滤 `..` 段、验证最终路径是否在允许的目录内。OpenClaw 同时使用了这三种方法。

HTTP 请求处理

`handleHttpRequest` 是整个文件服务的入口：

```
// src/canvas-host/server.ts — handleHttpRequest 核心逻辑

const handleHttpRequest = async (req, res) => {
  const url = new URL(req.url, "http://localhost");

  // 1. WebSocket 路径返回 426（需要升级）
  if (url.pathname === CANVAS_WS_PATH) {
    res.statusCode = liveReload ? 426 : 404;
    res.end(liveReload ? "upgrade required" : "not found");
    return true;
  }

  // 2. 基路径匹配（默认 /__openclaw__/canvas）
  let urlPath = url.pathname;
  if (basePath !== "/") {
    if (!urlPath.startsWith(`${basePath}/`) && urlPath !== basePath) {
      return false;  // 不属于 Canvas 的请求，交给下一个处理器
    }
    urlPath = urlPath.slice(basePath.length) || "/";
  }

  // 3. 只允许 GET/HEAD
  if (req.method !== "GET" && req.method !== "HEAD") {
    res.statusCode = 405;
    res.end("Method Not Allowed");
    return true;
  }

  // 4. 解析文件路径（含安全检查）
  const opened = await resolveFilePath(rootReal, urlPath);
  if (!opened) {
    res.statusCode = 404;
    res.end("not found");
    return true;
  }

  // 5. 读取文件、检测 MIME 类型
  const data = await opened.handle.readFile();
  const mime = lower.endsWith(".html")
    ? "text/html"
    : (await detectMime({ filePath: realPath })) ?? "application/octet-stream";

  // 6. HTML 文件注入 Live Reload 脚本
```

```
    res.setHeader("Cache-Control", "no-store");
    if (mime === "text/html") {
      const html = data.toString("utf8");
      res.end(liveReload ? injectCanvasLiveReload(html) : html);
      return true;
    }

    res.setHeader("Content-Type", mime);
    res.end(data);
    return true;
  };
```

几个关键设计点：

- **`Cache-Control: no-store`** ——禁用缓存，确保 Agent 更新文件后 WebView 立即获取最新版本
- **HTML 自动注入**——所有 HTML 响应都会被注入 Live Reload 脚本和跨平台动作桥
- **返回 `boolean`** —— `true` 表示已处理， `false` 表示不归 Canvas 管，由其他处理器继续

Live Reload：文件变更自动刷新

Live Reload 使用 **chokidar** 监视文件变更，通过 WebSocket 通知客户端刷新：

```
// src/canvas-host/server.ts — Live Reload 实现

// 1. 创建 WebSocket 服务器（noServer 模式，手动处理升级）
const wss = liveReload ? new WebSocketServer({ noServer: true }) : null;
const sockets = new Set<WebSocket>();
wss?.on("connection", ws => {
  sockets.add(ws);
  ws.on("close", () => sockets.delete(ws));
});

// 2. 广播刷新消息（带 75ms 防抖）
let debounce: NodeJS.Timeout | null = null;
const scheduleReload = () => {
  if (debounce) clearTimeout(debounce);
  debounce = setTimeout(() => {
    debounce = null;
    for (const ws of sockets) {
      ws.send("reload");  // 简单的文本消息
    }
  }, 75);
  debounce.unref?.();  // 不阻止进程退出
};

// 3. chokidar 监视文件变更
const watcher = chokidar.watch(rootReal, {
  ignoreInitial: true,                                  // 不触发已有文件
  awaitWriteFinish: { stabilityThreshold: 75, pollInterval: 10 },  // 等写入稳定
  ignored: [
    /(^|[\\/])\../,                                     // 忽略 dotfiles
    /(^|[\\/])node_modules([\\/]|$)/,                   // 忽略 node_modules
  ],
});
watcher.on("all", () => scheduleReload());
```

> **衍生解释——chokidar**：chokidar 是 Node.js 生态中最流行的文件监视库，是 `fs.watch` 的高级封装。它解决了原生 `fs.watch` 的诸多跨平台问题：macOS 上的 `FSEvents` 有时不报告文件名、Linux 上的 `inotify` 对递归监视支持有限、某些编辑器（如 Vim）会先删除再创建文件导致 watch 失效等。chokidar 统一了这些差异，提供可靠的跨平台文件监视。

Live Reload 的时序：

```
Agent 写入文件 → chokidar 检测变更 → 75ms 防抖等待
                                          |
                                  awaitWriteFinish (75ms)
                                          |
                                     scheduleReload()
                                          |
                                   75ms debounce 超时
                                          |
                              broadcastReload() → ws.send("reload")
                                          |
                                  客户端收到 "reload" → location.reload()
```

两个 75ms 延迟的含义不同：

- `awaitWriteFinish.stabilityThreshold: 75` ——等待文件写入完成（文件大小在 75ms 内不变才算完成）
- `setTimeout(broadcastReload, 75)` ——防抖：合并短时间内的多个文件变更为一次刷新

环境禁用

Canvas Host 在测试环境中自动禁用：

```
function isDisabledByEnv() {
  if (isTruthyEnvValue(process.env.OPENCLAW_SKIP_CANVAS_HOST)) return true;
  if (process.env.NODE_ENV === "test") return true;
  if (process.env.VITEST) return true;
  return false;
}
```

禁用时返回一个空壳 Handler，所有方法都是 no-op：

```
if (isDisabledByEnv() && opts.allowInTests !== true) {
  return {
    rootDir: "",
    handleHttpRequest: async () => false,
    handleUpgrade: () => false,
    close: async () => {},
  };
}
```

独立服务器模式

`startCanvasHost()` 在 Handler 之上创建一个独立的 HTTP 服务器：

```
// src/canvas-host/server.ts — startCanvasHost（简化版）

export async function startCanvasHost(opts): Promise<CanvasHostServer> {
  const handler = opts.handler ?? await createCanvasHostHandler({...});
  const bindHost = opts.listenHost || "0.0.0.0";

  const server = http.createServer((req, res) => {
    void (async () => {
      // 优先级：A2UI 请求 > Canvas 请求 > 404
      if (await handleA2uiHttpRequest(req, res)) return;
      if (await handler.handleHttpRequest(req, res)) return;
      res.statusCode = 404;
      res.end("Not Found");
    })();
  });

  // WebSocket 升级处理
  server.on("upgrade", (req, socket, head) => {
    if (handler.handleUpgrade(req, socket, head)) return;
    socket.destroy();  // 不认识的升级请求直接断开
  });

  // 监听端口（0 表示操作系统自动分配）
  const listenPort = opts.port > 0 ? opts.port : 0;
  server.listen(listenPort, bindHost);

  return { port: boundPort, rootDir: handler.rootDir, close: ... };
}
```

请求优先级链：**A2UI → Canvas → 404**。A2UI 路径（`/__openclaw__/a2ui`）优先于 Canvas 路径（`/__openclaw__/canvas`），因为 A2UI 是 Canvas 之上的功能层。

25.2.2 A2UI 核心（`src/canvas-host/a2ui.ts` 与 `a2ui/`）

路径常量

A2UI 模块定义了三个关键路径：

```
// src/canvas-host/a2ui.ts

export const A2UI_PATH       = "/__openclaw__/a2ui";     // A2UI 渲染器资源
export const CANVAS_HOST_PATH = "/__openclaw__/canvas";   // Canvas 文件服务
export const CANVAS_WS_PATH   = "/__openclaw__/ws";       // Live Reload WebSocket
```

所有路径都以 `/__openclaw__/` 为前缀，这是一个不太可能与用户内容冲突的命名空间。

A2UI 根目录发现

A2UI 渲染器的资源文件（`index.html` + `a2ui.bundle.js`）可能位于不同位置，取决于运行方式：

```
// src/canvas-host/a2ui.ts — resolveA2uiRoot

async function resolveA2uiRoot(): Promise<string | null> {
  const here = path.dirname(fileURLToPath(import.meta.url));
  const candidates = [
    path.resolve(path.dirname(process.execPath), "a2ui"),  // 可执行文件旁
    path.resolve(here, "a2ui"),                             // 源码/dist 同级
    path.resolve(here, "../../src/canvas-host/a2ui"),       // dist → 源码回退
    path.resolve(process.cwd(), "src/canvas-host/a2ui"),    // 仓库根目录
    path.resolve(process.cwd(), "dist/canvas-host/a2ui"),   // 仓库 dist 目录
  ];

  for (const dir of candidates) {
    try {
      // 必须同时存在 index.html 和 a2ui.bundle.js
      await fs.stat(path.join(dir, "index.html"));
      await fs.stat(path.join(dir, "a2ui.bundle.js"));
      return dir;
    } catch { /* 尝试下一个 */ }
  }
  return null;
}
```

发现策略的优先级反映了不同的部署场景：

优先级	候选路径	场景
1	`process.execPath` 旁	打包为单文件可执行程序
2	当前模块同级 `a2ui/`	源码直接运行或 dist 已复制资源
3	`../../src/canvas-host/a2ui`	dist 运行但未复制资源，回退到源码
4	`cwd()/src/canvas-host/a2ui`	从仓库根目录运行
5	`cwd()/dist/canvas-host/a2ui`	从仓库根目录运行 dist

发现结果会被缓存（`cachedA2uiRootReal`），并通过 Promise 去重（`resolvingA2uiRoot`）避免并发时重复搜索。

A2UI HTTP 请求处理

`handleA2uiHttpRequest` 处理 `/__openclaw__/a2ui` 路径下的请求：

```typescript
// src/canvas-host/a2ui.ts — handleA2uiHttpRequest（简化版）

export async function handleA2uiHttpRequest(req, res): Promise<boolean> {
  const url = new URL(req.url, "http://localhost");

  // 路径匹配
  if (!url.pathname.startsWith(`${A2UI_PATH}/`) && url.pathname !== A2UI_PATH) {
    return false;  // 不是 A2UI 请求
  }

  // 只允许 GET/HEAD
  if (req.method !== "GET" && req.method !== "HEAD") {
    res.statusCode = 405;
    res.end("Method Not Allowed");
    return true;
  }

  // 查找 A2UI 资源根目录
  const a2uiRootReal = await resolveA2uiRootReal();
  if (!a2uiRootReal) {
    res.statusCode = 503;
    res.end("A2UI assets not found");
    return true;
  }

  // 解析文件路径（含安全检查）
  const rel = url.pathname.slice(A2UI_PATH.length);
  const filePath = await resolveA2uiFilePath(a2uiRootReal, rel || "/");

  // HTML 文件注入 Live Reload
  if (mime === "text/html") {
    const html = await fs.readFile(filePath, "utf8");
    res.end(injectCanvasLiveReload(html));
    return true;
  }

  res.end(await fs.readFile(filePath));
  return true;
}
```

A2UI 的文件服务与 Canvas 共享相同的安全策略：拒绝 `..` 段、拒绝符号链接、验证 `realpath` 在根目录内。

Live Reload 注入

`injectCanvasLiveReload()` 是 Canvas 和 A2UI 共享的关键函数，它将跨平台动作桥和 Live Reload 客户端注入到每个 HTML 响应中：

```
// src/canvas-host/a2ui.ts — injectCanvasLiveReload

export function injectCanvasLiveReload(html: string): string {
  const snippet = `<script>
(() => {
  // === 跨平台动作桥 ===
  function postToNode(payload) {
    const raw = JSON.stringify(payload);
    // iOS: webkit.messageHandlers
    // Android: window.openclawCanvasA2UIAction
    ...
  }
  function sendUserAction(userAction) {
    const id = userAction.id || crypto.randomUUID();
    return postToNode({ userAction: { ...userAction, id } });
  }
  globalThis.OpenClaw = { postMessage: postToNode, sendUserAction };
  globalThis.openclawSendUserAction = sendUserAction;

  // === Live Reload WebSocket ===
  const proto = location.protocol === "https:" ? "wss" : "ws";
  const ws = new WebSocket(proto + "://" + location.host + "/__openclaw__/ws");
  ws.onmessage = (ev) => {
    if (ev.data === "reload") location.reload();
  };
})();
</script>`;

  // 优先插入到 </body> 之前
  const idx = html.toLowerCase().lastIndexOf("</body>");
  if (idx >= 0) {
    return html.slice(0, idx) + "\n" + snippet + "\n" + html.slice(idx);
  }
  // 没有 </body> 标签就追加到末尾
  return html + "\n" + snippet + "\n";
}
```

注入的脚本做了两件事：

1. **动作桥**：暴露 `window.OpenClaw.sendUserAction()` 和 `window.openclawSendUserAction()` 全局函数
2. **Live Reload**：建立到 `/__openclaw__/ws` 的 WebSocket 连接，收到 `"reload"` 消息时自动刷新页面

25.2.3 A2UI 打包（`scripts/bundle-a2ui.sh`）

A2UI 渲染器的实际 UI 实现由两部分组成：

组件	路径	技术栈
Lit 渲染器	`vendor/a2ui/renderers/lit`	Lit Web Components
应用逻辑	`apps/shared/OpenClawKit/Tools/CanvasA2UI`	TypeScript + Rolldown

最终打包为一个 `a2ui.bundle.js` 文件，放置在 `src/canvas-host/a2ui/` 目录下。

打包流程

```
#!/usr/bin/env bash
# scripts/bundle-a2ui.sh

set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
HASH_FILE="$ROOT_DIR/src/canvas-host/a2ui/.bundle.hash"
OUTPUT_FILE="$ROOT_DIR/src/canvas-host/a2ui/a2ui.bundle.js"
A2UI_RENDERER_DIR="$ROOT_DIR/vendor/a2ui/renderers/lit"
A2UI_APP_DIR="$ROOT_DIR/apps/shared/OpenClawKit/Tools/CanvasA2UI"
```

增量构建：哈希校验

打包脚本使用 **SHA-256 哈希校验** 实现增量构建——只在源文件实际变更时才重新打包：

```bash
# 1. 收集输入文件列表
INPUT_PATHS=(
  "$ROOT_DIR/package.json"
  "$ROOT_DIR/pnpm-lock.yaml"
  "$A2UI_RENDERER_DIR"       # 整个 Lit 渲染器目录
  "$A2UI_APP_DIR"            # 整个应用逻辑目录
)

# 2. 计算当前哈希（内嵌 Node.js 脚本递归遍历所有文件）
current_hash="$(compute_hash)"

# 3. 与上次哈希比较
if [[ -f "$HASH_FILE" ]]; then
  previous_hash="$(cat "$HASH_FILE")"
  if [[ "$previous_hash" == "$current_hash" && -f "$OUTPUT_FILE" ]]; then
    echo "A2UI bundle up to date; skipping."
    exit 0
  fi
fi
```

`compute_hash` 函数内嵌了一段 Node.js 脚本，递归遍历所有输入文件，对每个文件的相对路径和内容计算 SHA-256 哈希。哈希结果存储在 `.bundle.hash` 文件中。

Docker 环境处理

在 Docker 构建中， `vendor/` 和 `apps/` 目录可能被 `.dockerignore` 排除。脚本对此有特殊处理：

```bash
# Docker builds exclude vendor/apps via .dockerignore.
# In that environment we must keep the prebuilt bundle.
if [[ ! -d "$A2UI_RENDERER_DIR" || ! -d "$A2UI_APP_DIR" ]]; then
  echo "A2UI sources missing; keeping prebuilt bundle."
  exit 0
fi
```

这意味着 `a2ui.bundle.js` 被视为**受版本控制的预构建产物**——提交到仓库中，Docker 构建直接使用，不需要重新打包。

构建工具链

实际的构建分两步：

```
# 步骤 1：TypeScript 编译 Lit 渲染器
pnpm -s exec tsc -p "$A2UI_RENDERER_DIR/tsconfig.json"

# 步骤 2：Rolldown 打包应用逻辑为单文件
rolldown -c "$A2UI_APP_DIR/rolldown.config.mjs"

# 步骤 3：保存哈希
echo "$current_hash" > "$HASH_FILE"
```

> **衍生解释——Rolldown**：Rolldown 是一个用 Rust 编写的 JavaScript 打包工具（bundler），旨在成为 Rollup 的高性能替代品。它与 Rollup 共享配置格式（`rolldown.config.mjs`），但编译速度快得多。OpenClaw 选择 Rolldown 将 A2UI 的多个模块打包为一个 `a2ui.bundle.js` 文件，使其可以在 WebView 中通过单个 `<script src="a2ui.bundle.js">` 加载。

> **衍生解释——Lit**：Lit 是 Google 开发的轻量级 Web Components 库。它基于浏览器原生的 Custom Elements 和 Shadow DOM 标准，提供了响应式属性、模板字面量（`html\...`）等便捷 API。OpenClaw 的 A2UI 渲染器使用 Lit 来实现 `自定义元素，将 JSONL 命令解析为实际的 DOM 元素。

打包产物在架构中的位置

```
Agent 发送 JSONL
    │
    ↓
原生应用 WebView 加载:
    /__openclaw__/a2ui
        │
        ├── index.html      ← A2UI 宿主页面
        └── a2ui.bundle.js  ← 打包产物（本节描述的构建结果）
                │
                └── 包含:
                    ├── Lit 渲染器（vendor/a2ui/renderers/lit）
                    │   解析 JSONL → 创建 Web Components
                    └── 应用逻辑（apps/shared/.../CanvasA2UI）
                        状态管理、布局计算、主题等
```

本节小结

1. Canvas Host 采用**两层架构**： `CanvasHostHandler` （可嵌入）和 `CanvasHostServer` （独立运行），Handler 可以被嵌入 Gateway 的 HTTP 层
2. **静态文件服务**从 `~/.openclaw/canvas/` 目录提供内容，通过三重路径安全机制（URL 规范化、 `..` 检测、 `openFileWithinRoot` ）防止路径遍历攻击
3. **Live Reload** 使用 chokidar 监视文件 + WebSocket 广播 + 75ms 防抖，实现文件变更时 WebView 自动刷新
4. 所有 HTML 响应都会被**自动注入**跨平台动作桥（iOS/Android JS Bridge）和 Live Reload 客户端
5. A2UI 根目录通过**多候选路径搜索**定位，支持源码运行、dist 运行、打包为可执行文件等多种部署场景
6. A2UI 打包使用 **SHA-256 增量构建**——通过哈希校验跳过未变更的构建，Docker 环境使用预构建产物
7. 构建工具链为 **tsc**（TypeScript 编译）+ **Rolldown**（Rust 打包器），产出单文件 `a2ui.bundle.js`